

Laboratorio 3 e 6

Anna Carelli (1029621) e Ilaria Consoli (944548)

Introduzione

Il laboratorio consiste nella creazione di un dataset stratificato focalizzato su termini polisemici con doppia accezione (comune e medica). Nello specifico è stato costruito un dataset composto da termini inglesi che rispettassero questa condizione e in seguito ne è stata calcolata l'entropia costruendo 5 quintili basandosi su di essa. Utilizzando un modello ollama in locale sono state generate delle frasi d'esempio sia per il caso comune che per quello medico.

Nella seconda parte del progetto si applicano tre diversi algoritmi di disambiguazione al dataset:

1. Most Frequent Sense (baseline)
2. Lesk
3. Algoritmo basato su embedding

Una volta verificata la performance sul dataset su frasi comuni e mediche, si analizzano i risultati su questi aspetti:

- Correlazione tra entropia e accuratezza dei risultati
- Confronto tra frasi con significato comune e medico
- Differenza tra i tre algoritmi

Glossario medico

I termini selezionati sono stati recuperati da vari articoli scientifici o pensati da noi. Riportiamo di seguito l'elenco dei termini medici.

Termine	Definizione medica	Definizione comune
Discharge	any of several bodily processes by which substances go out of the body	the act of venting
Shock	(pathology) bodily collapse or near collapse caused by inadequate oxygen delivery to the cells; characterized by reduced cardiac output and rapid heartbeat and circulatory insufficiency and pallor	the feeling of distress and disbelief that you have when something bad happens accidentally
Stroke	a sudden loss of consciousness resulting when the rupture or occlusion of a blood vessel leads to oxygen lack in the brain	(sports) the act of swinging or striking at a ball with a club or racket or bat or cue or hand
Block	block passage through	a solid piece of something (usually having flat rectangular sides)
Cast	bandage consisting of a firm covering (often made of plaster of Paris) that immobilizes	the actors in a play

Termine	Definizione medica	Definizione comune
	broken bones while they heal	
Depression	a state of depression and anhedonia so severe as to require clinical intervention	a long-term economic state characterized by unemployment and low prices and low levels of trade and investment
Head	the upper part of the human body or the front part of the body in animals; contains the face and brains	a person who is in charge
Cold	a mild viral infection involving the nose and respiratory passages (but not the lungs)	the sensation produced by low temperatures
Culture	(biology) the growing of microorganisms in a nutrient medium (such as gelatin or agar)	a particular society at a particular time and place
Index	the finger next to the thumb	a numerical scale used to compare variables with one another or with some reference number
Labor	concluding state of pregnancy; from the onset of contractions to the birth of a child	a social class comprising those who do manual labor or work for wages
Arch	a curved bony structure supporting or enclosing organs (especially the inner sides of the feet)	a curved shape in the vertical plane that spans an opening
Arrest	absence of systole; failure of the ventricles of the heart to contract (usually caused by ventricular fibrillation) with consequent absence of the heart beat leading to oxygen lack and eventually to death	the act of apprehending (especially apprehending a criminal)
Negative	not indicating the presence of microorganisms or disease or a specific condition	a reply of denial
Vessel	a tube in which a body fluid circulates	a craft designed for water transportation
Valve	a structure in a hollow organ (like the heart) with a flap to insure one-way flow of fluid through it	control consisting of a mechanical device for controlling the flow of a fluid
Tube	(anatomy) any hollow cylindrical body structure	conduit consisting of a long hollow object (usually cylindrical) used to hold and conduct objects or liquids or gases
Cage	the bony enclosing wall of the chest	an enclosure made of wire or metal bars in which birds or animals can be kept

Termine	Definizione medica	Definizione comune
Leg	a human limb; commonly used to refer to a whole limb but technically only the part of the limb between the knee and ankle	one of the supports for a piece of furniture
Pupil	the contractile aperture in the center of the iris of the eye; resembles a large black dot	a learner who is enrolled in an educational institution
Patient	a person who requires medical care	enduring trying circumstances with even temper or characterized by such endurance

Le definizioni fornite sono state prese direttamente da WordNet, considerando i rispettivi synset. Tutti i synset sono stati riportati nel file *Dataset_medico.csv*.

Strumenti utilizzati

Per il codice abbiamo utilizzato le seguenti librerie:

- *pandas*: utilizzata per caricare, manipolare e gestire i dataset strutturati in formato tabellare.
- *nltk*: impiegata per l'elaborazione del linguaggio naturale, l'estrazione di synset da *wordnet* e la disambiguazione tramite *lesk*.
- *math*: utilizzata per eseguire i calcoli matematici logaritmici necessari a ottenere l'entropia dei termini.
- *ollama*: per l'interazione con il modello LLM locale al fine di generare le frasi di contesto.
- *matplotlib.pyplot*: per la generazione e il salvataggio dei grafici sull'accuratezza dei risultati.
- *seaborn*: utilizzata per realizzare visualizzazioni statistiche avanzate come i grafici a densità e a barre in pila.
- *spacy*: per consentire gli embedding nel WSD
- *ewiser (1)*: package per l'implementazione un algoritmo WSD basato su embedding. Lo utilizziamo come plug in della pipeline nlp di spacy

Per eseguire la seconda parte del progetto, abbiamo utilizzato un ambiente virtuale su *conda*.

Implementazione

Calcolo entropia e quintili

Questa prima parte del progetto consiste nel calcolo dell'entropia di ogni termine tramite la formula vista a lezione:

$$H = - \sum (p_i * \log_2(p_i))$$

In cui ho p_i che rappresenta la probabilità del singolo lemma, calcolata come la frequenza del lemma diviso il totale dei conteggi.

Ecco l'implementazione nel codice:

Python

```
def calculate_entropy(word):
    word = word.strip().lower()
    lemmas = wn.lemmas(word)
    if not lemmas:
        return 0.0
    counts = [lemma.count() for lemma in lemmas]
    total = sum(counts)
    entropy = 0.0
    for count in counts:
        if count > 0:
            p = count / total
            entropy -= p * math.log2(p)
    return entropy
```

Si applica la formula dell'entropia su ogni termine e di seguito riportiamo il codice per la divisione in 5 quintili di entropia dopo ad aver filtrato i casi irrilevanti (con Entropia = 0). Per creare i quintili abbiamo utilizzato la funzione di pandas *qcut*: si indicano i valori da utilizzare, il numero di quintili e le label per ognuno. Infine ordiniamo i termini sulla base dei quintili.

Python

```
# Applicazione funzione entropia e creazione della colonna
df['Entropy'] = df['English'].apply(calculate_entropy)

# Arrotondamento dei valori a 2 cifre decimali e esclusione dei termini a entropia 0
df['Entropy'] = df['Entropy'].round(2)
df_active = df[df['Entropy'] > 0]

# Creazione colonna quintili e applicazione funzione pandas dedicata
df_active['Quintile'] = pd.qcut(df_active['Entropy'], 5, labels=[1,2,3,4,5])

# Ordinamento in base ai quintili
df_ordered = df_active.sort_values(by='Entropy', ascending=False)
```

I risultati di questa operazione sono salvati nel file *Dataset_medico_con_Entropia.csv*.
Brevemente riportiamo i valori riassunti:

Termine	Entropia	Quintile
Discharge	2.83	5
Shock	2.71	5
Stroke	2.57	5
Block	2.03	5
Cast	1.92	4
Depression	1.87	4
Head	1.86	4
Cold	1.63	4
Culture	1.61	3
Index	1.58	3
Labor	1.51	3
Arch	1.37	3
Arrest	1.23	2
Negative	1.19	2
Vessel	1.0	2
Valve	0.92	1
Tube	0.92	1
Cage	0.92	1
Leg	0.66	1
Pupil	0.5	1
Patient	0.24	1

Generazione delle frasi di esempio

La generazione è stata implementata con il modello *phi3* scaricato in locale tramite la libreria *ollama*. Il prompt fornisce sia il termine che la definizione da utilizzare. Il codice è riportato di seguito.

Python

```
for i in range(df_ordered.shape[0]):
    # Prendo termini e synset necessari come input e contesto
    word = df_ordered.iloc[i]['English']
    synset_medical = wn.synset(df_ordered.iloc[i]['Medical'])
    synset_common = wn.synset(df_ordered.iloc[i]['Common'])

    # Prompt utilizzato per la generazione puntuale in ambito medico
    prompt = f"Write a natural English sentence using the word '{word}'. It MUST be used in this exact sense: '{synset_medical.definition()}'. Output ONLY the sentence, nothing else."

    # Generazione frase in ambito medico
    response = ollama.chat(model='phi3', messages=[
        {'role': 'user', 'content': prompt}
    ])
    sentence = response['message']['content'].strip('')

    # Stessa cosa ma nel contesto comune
    prompt_common = f"Write a natural English sentence using the word '{word}'. It MUST be used in this exact sense: '{synset_common.definition()}'. Output ONLY the sentence, nothing else."
    response_common = ollama.chat(model='phi3', messages=[
        {'role': 'user', 'content': prompt_common}
    ])
    sentence_common = response_common['message']['content'].strip('')

    # Aggiunta al dataframe di destinazione
    df_prompt.loc[len(df_prompt)] = [word, synset_medical.name(),
    synset_common.name(), sentence, sentence_common,
    df_ordered.iloc[i]['Quintile']]
```

Il dataset finale è salvato come *Benchmark.csv*.

Verifica qualità dataset con WSD

Per testare la bontà del dataset delle frasi d'esempio, utilizziamo tre diversi algoritmi di disambiguazione per avere più metri di paragone. In particolare:

- Most Frequent Sense (MFS): prende il synset più utilizzato del termine su WordNet.
- Algoritmo di Lesk: utilizza il contesto di tutta la frase per definire il synset più vicino
- Algoritmo basato su embedding di Ewiser: utilizza i pesi ottenuti dal

Di seguito riportiamo il codice dei tre metodi applicati su entrambi i tipi di frase:

Python

```
def MFS(df):
    synsets = []
    for index, row in df.iterrows():
        word = row['Term']
        synset = wn.synsets(word)[0] if wn.synsets(word) else None
        synsets.append(synset)
    return synsets
```

Most Frequent Sense trova un senso a parola perché non dipende dalla frase ma solo dal termine stesso.

Python

```
def lesk_algorithm(df):
    synsets_medical = []
    synsets_common = []
    for index, row in df.iterrows():
        word = row['Term']
        sentence_medical = row['Sentence Medical']
        sentence_common = row['Sentence Common']
        synset_medical = lesk(sentence_medical.split(), word)
        synset_common = lesk(sentence_common.split(), word)
        synsets_medical.append(synset_medical)
        synsets_common.append(synset_common)
    return synsets_medical, synsets_common
```

Nel caso di **Lesk**, applichiamo il metodo lesk della libreria nltk per trovare i synset.

Python

```
from ewiser.spacy.disambiguate import Disambiguator

nlp = spacy.load("en_core_web_sm", disable=['parser', 'ner'])
weights = "C:/Users/Anna/Desktop/UNI/TLN/Di Caro/ewiser/ewiser.semcor_base.pt"
wsd = Disambiguator(weights, lang="en")
nlp.add_pipe(wsd, last=True)

def embedding(df):
    synsets_medical = []
    synsets_common = []
    for index, row in df.iterrows():
        word = row['Term']
        sentence_medical = row['Sentence Medical']
        sentence_common = row['Sentence Common']
```

```

docmed = nlp(sentence_medical)
doccom = nlp(sentence_common)
for token in docmed:
    if token.text.lower().strip() == word.lower().strip():
        synset_medical = token._.offset
        break

for token in doccom:
    if token.text.lower().strip() == word.lower().strip():
        synset_common = token._.offset
        break

synsets_medical.append(offset_to_synset(synset_medical))
synsets_common.append(offset_to_synset(synset_common))
return synsets_medical, synsets_common

```

Per l'ultimo metodo, abbiamo utilizzato il plug-in di spaCy chiamato **EWISER (Enhanced WSD Integrating Synset Embeddings and Relations)**. Viene da un repository universitario di github. Seguendo i requisiti e l'iter di installazione, la difficoltà maggiore è stata la necessità di eseguire il tutto all'interno dell'ambiente virtuale su *conda* per rispettare le versioni di python e pytorch richieste dal package.

Quando si passa una frase alla pipeline estesa di ewiser, il modulo fa scorrere le parole attraverso la sua rete neurale pre-addestrata (guidata dal file .pt dei pesi). Questa rete cattura le sfumature semantiche della frase e, appoggiandosi alle relazioni di dizionari come WordNet, calcola matematicamente quale sia il significato più probabile per ogni singolo termine.

Il synset trovato viene restituito come un codice identificativo offset. Tramite la funzione seguente lo abbiamo convertito in un synset vero e proprio.

Python

```

def offset_to_synset(offset):
    if not isinstance(offset, str) or not offset.startswith('wn:'):
        return None

    codice = offset.replace('wn:', '')
    pos = codice[-1]
    offset_num = int(codice[:-1])
    try:
        return wn.synset_from_pos_and_offset(pos, offset_num)
    except:
        return None

```

I risultati ottenuti sono stati salvati in un altro dataset per le analisi successive.

Analisi dei risultati

Calcoliamo l'accuratezza degli output dei tre algoritmi tramite la seguente funzione *evaluate*. Restituisce un valore globale di accuratezza e un array di 5 misure di precisione per quintile.

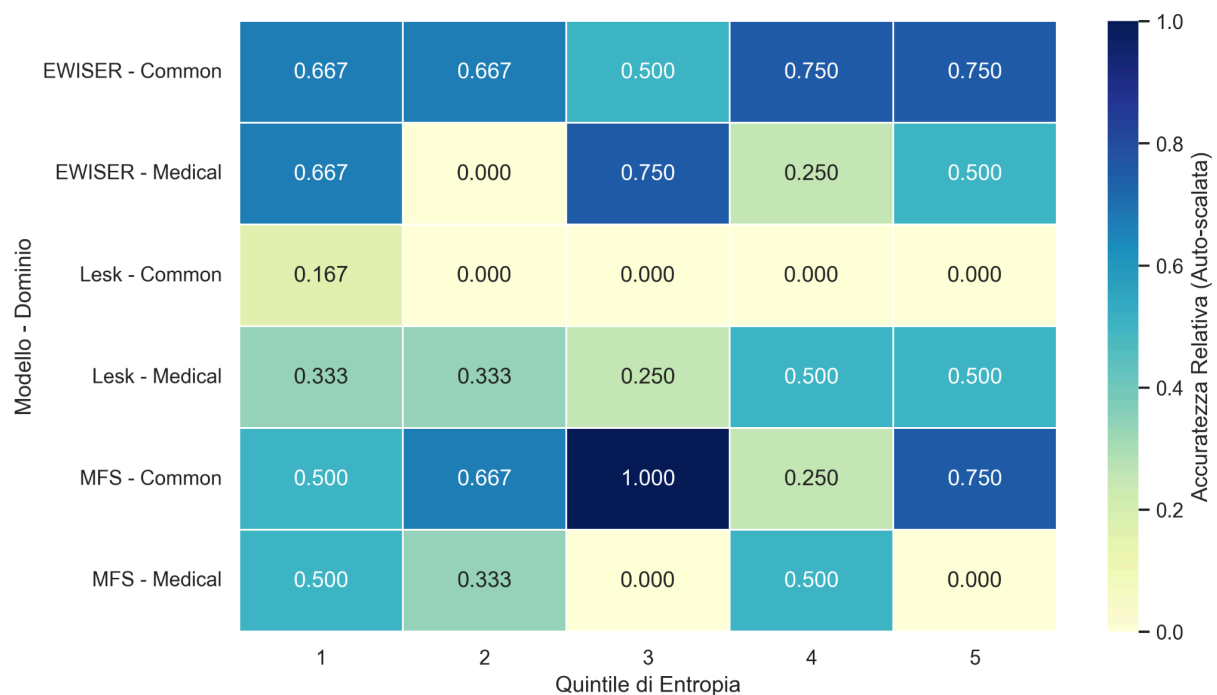
Python

```
def evaluate(synsets_predicted, synsets_true):
    corrects = 0
    corrects_quintile = [0,0,0,0,0]
    len_quintile = [0,0,0,0,0]
    for i in range(len(synsets_predicted)):
        len_quintile[df_bench['Quintile'][i]-1] += 1
        # Controlliamo i casi nulli di sysnet non trovati
        if hasattr(synsets_predicted[i], 'name'):
            if synsets_predicted[i].name() == synsets_true[i]:
                corrects += 1
                corrects_quintile[df_bench['Quintile'][i] - 1] += 1
    return corrects / len(synsets_predicted), [corrects_quintile[i] / len_quintile[i] for i in range(5)]
```

Per visualizzare chiaramente i valori e aiutarci nell'analisi, abbiamo creato due grafici utilizzando *seaborn*.

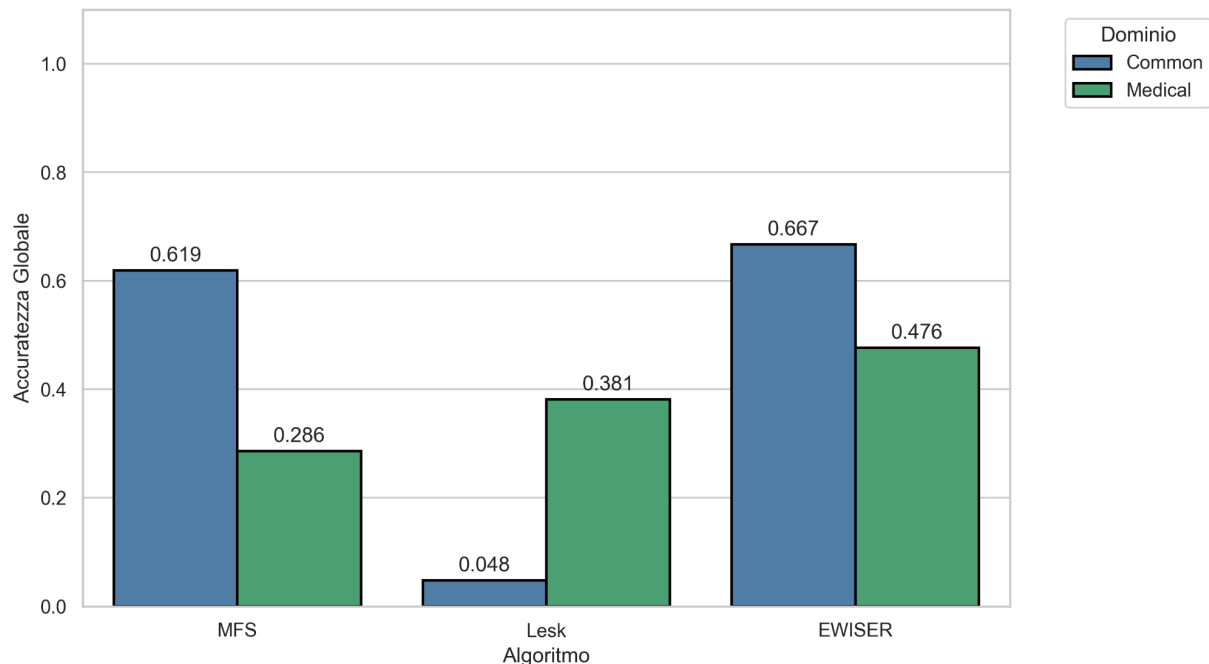
Correlazione tra entropia e accuratezza

HeatMap per la correlazione tra i quintili di entropia e l'accuratezza della disambiguazione



Accuratezza globale divisa per algoritmo e dominio

Boxplot per confrontare in base all'algoritmo l'accuratezza ottenuta globalmente senza considerare i quintili.



Conclusioni

Come possiamo osservare dal primo grafico, i dati mostrano che l'aumento dell'entropia (ovvero il livello di ambiguità teorica di un termine) non si traduce in un automatico degrado delle performance per tutti i modelli. Il comportamento **varia in base all'architettura**: il modello basato su embedding dimostra una notevole resilienza, mantenendo un'accuratezza accettabile e una buona stabilità anche nei quintili a maggiore entropia. Al contrario, algoritmi più rigidi come MFS mostrano una forte instabilità sensibile al contesto, alternando picchi di precisione a fallimenti totali in specifici quintili del dominio medico. Dunque l'entropia non agisce come una variabile isolata, ma il suo impatto è fortemente variato dal tipo di algoritmo e dal dominio in cui la parola è inserita.

Come ci aspettavamo, il dominio della frase esercita un'influenza determinante sulle capacità predittive dei modelli.

- Nel **dominio comune**, i sistemi beneficiano della generalità del linguaggio: EWISER raggiunge qui il suo picco di accuratezza ed MFS dimostra una valida alternativa dato che il significato non medico è spesso quello più frequente, mentre Lesk fallisce quasi interamente.
- Nel **dominio medico**, emerge la forte tendenza di MFS verso il linguaggio comune, che ne causa un crollo delle prestazioni. Al contrario, l'algoritmo di Lesk sperimenta un netto miglioramento, probabilmente perché il linguaggio tecnico usato è più preciso, standardizzato e ha più parole in comune con le definizioni dei dizionari.

In conclusione, analizziamo i tre algoritmi:

- **EWISER:** Rappresenta l'approccio più avanzato. grazie alla comprensione del contesto offerta dalle reti neurali e dagli embedding, supera la rigidità dei metodi tradizionali e si conferma il modello più bilanciato ed efficiente in entrambi i domini.
- **MFS:** Si rivela una baseline efficace ma priva di flessibilità: fallisce sistematicamente con linguaggi specialistici perché ignora completamente il contesto.
- **Lesk:** Basandosi sul semplice overlap lessicale, va in difficoltà con il linguaggio troppo generico, ma si rivela utile in ambito medico, dove il vocabolario è più tecnico e standardizzato.

Il nostro dataset è stato creato per mettere in luce la polisemia di significati molto lontani come quello medico e quello comune. I tre modelli WSD hanno provato la qualità del nostro dataset secondo diversi aspetti, dimostrando con MFS la lontananza dei due significati e con Ewiser la buona produzione di frasi contestualizzate.

Bibliografia

1. EWISER (Enhanced WSD Integrating Synset Embeddings and Relations) - <https://github.com/SapienzaNLP/ewiser/tree/master/ewiser>